

Практическая работа «Методы цифровой обработки изображений на основе сверточных нейронных сетей»

Цель: научиться применять сверточные нейронные сети для автоматического выделения и классификации объектов на цифровых изображениях в соответствии с заданными классами.

Содержание отчета: ответы на контрольные вопросы, выполненные задания и вывод о проделанной работе.

Теоретическая часть

Обучение ИНС предполагает определение весов соединений между нейронами таким образом, чтобы ИНС приближала необходимую функцию с заданной точностью. Применяются три подхода к обучению ИНС: обучение с учителем (supervised learning), обучение без учителя (unsupervised learning) и обучение с подкреплением (reinforcement learning). При выполнении данной работы необходимо применить первый подход, т. е. обучение с учителем, который предполагает, что имеется обучающее множество с заранее известными правильными ответами. Для заданных весовых коэффициентов ИНС сравнивается полученный результат с правильным ответом и в зависимости от полученной ошибки принимается решение для корректировки весов [13].

Формально общая постановка задачи обучения с учителем представляется следующим образом. Пусть имеется обучающее множество, состоящее из N примеров. Каждый обучающий пример задаётся в следующем виде: $\{(x_1, y_1), \dots, (x_N, y_N)\}$, где x_i – вектор входных признаков i -го примера, y_i – вектор, задающий выходное значение i -го примера.

Тогда алгоритм такого обучения должен определить функцию $g: X \rightarrow Y$, где X – пространство входов модели, Y – пространство выходов. Функция g является элементом пространства функций G , которое называют также пространством гипотез.

Существует два основных подхода к выбору функции g : минимизация эмпирического риска и минимизация структурного риска. Минимизация эмпирического риска ищет функцию, которая наилучшим образом соответствует обучающим данным. Минимизация структурного риска включает в себя функцию штрафа, которая ищет компромисс между смещением и дисперсией (дилемма смещение-дисперсия – чем меньше смещение оценки параметра

модели, тем выше её дисперсия, и наоборот). В обоих случаях предполагается, что обучающее множество состоит из независимых и одинаково распределенных пар (x_i, y_i) . С целью проверить, насколько хорошо функция соответствует обучающим данным, определяется функция потерь (целевая функция, функция ошибки, функция затрат или функция стоимости) $L(y_i, \hat{y})$, где $\hat{y}$ – значение, предсказанное моделью для примера (x_i, y_i) , которая может быть определена как разница между желаемым выходным значением y и прогнозируемым выходным значением $\hat{y}$.

Риск $R(g)$ определяется как потери g которые на обучаемых данных могут быть оценены следующим образом:

$$R(g) = \frac{1}{N} \sum_{i=1}^N L(y_i, g(x_i)).$$

Для обучения сверточных нейронных сетей часто используется метод обратного распространения ошибки. Общий принцип метода состоит из трех этапов: прямой проход (вычисление градиентов), вычисление значения функции ошибки (функции потерь) и ее градиента, обратный проход (коррекция весовых коэффициентов) [14]. Коррекция весовых коэффициентов направлена на минимизацию значения функции потерь, и наиболее популярным методом, определяющим минимальное значение функции потерь, является градиентный спуск [15].

На первом этапе обучения необходимо инициализировать веса ИНС и определить функцию потерь, т. к. на результаты обучения большое влияние оказывает подбор начальных значений весов ИНС. Идеальными считаются значения, достаточно близкие к оптимальным [16], однако определить их практически невозможно для задачи обнаружения объектов на изображении. Корректный выбор весов позволяет предотвратить резкое насыщение (взрыв) или исчезновения выходов активационного слоя во время прямого прохода через глубокую, т. е. с большим количеством слоев, ИНС. Это приведет к тому, что либо потребуются большее количество этапов обучения (эпох) для достижения необходимой точности сети, либо в принципе невозможно будет обучить ИНС. Пока не существует универсального метода подбора весов, который может гарантировать нахождение наилучшей начальной точки для любой решаемой задачи, однако получены и применяются следующие рекомендации:

- не использовать нулевые начальные веса. В противном случае все нейроны будут вести себя одинаково и сеть не сможет обучиться для разных признаков;

— исходные значения весов должны быть ассиметричны, т. е. два скрытых блока с одинаковой функцией активации, подключенные к одним и тем же входам, должны иметь разные начальные параметры;

— случайная инициализация считается универсальным способом установления начальных значений весам сети. Она должна обеспечивать такую стартовую точку активации нейронов, которая лежала бы достаточно далеко от зоны насыщения. Для этого диапазон допустимых значений ограничивается. Может быть использован алгоритм, который позволяет формировать массив, элементами которого являются случайные величины, распределённые по нормальному закону с математическим ожиданием, равным нулю, и среднеквадратичным отклонением, равным единице. Однако особенность этого подхода в том, что он удовлетворительно работает при обучении небольших архитектур ИНС, но результативность для глубоких ИНС ухудшается.

Современные библиотеки машинного обучения используют вариации инициализации весов маленькими случайными числами с разными способами распределения случайных чисел.

В качестве функции потерь наиболее часто используют следующие:

— средняя квадратичная ошибка (MSE). Является наиболее распространенной и широко используется в линейной регрессии, в которой зависимость переменной от одной или нескольких других переменных определяется линейной функцией, в качестве показателя эффективности. Для расчёта используется формула

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2,$$

где y_i — фактический ожидаемый результат, $\hat{y}_i$ — прогноз модели;

— среднеквадратическая ошибка (RMSE):

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2} = \sqrt{MSE};$$

— средняя абсолютная ошибка (MAE). Представляет собой усредненную сумму модулей разницы между реальными и предсказанными значениями:

$$MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|;$$

– среднеквадратичная логарифмическая ошибка (RMSLE). Представляет собой RMSE, рассчитанную в логарифмическом масштабе:

$$\begin{aligned} RMSLE &= \sqrt{\frac{1}{m} \sum_{i=1}^m (\log(y_i + 1) - \log(\hat{y}_i + 1))^2} = RMSE(\log(y_i + 1), \log(\hat{y}_i + 1)) = \\ &= \sqrt{MSE(\log(y_i + 1), \log(\hat{y}_i + 1))}; \end{aligned}$$

– кросс-энтропия (или логарифмическая функция потерь) измеряет расхождение между двумя вероятностными распределениями. Чем меньше значение кросс-энтропии, тем больше распределения похожи друг на друга. Кросс энтропия определяется как

$$H(P, Q) = - \sum_x P(x) \log Q(x);$$

где P – распределение истинных ответов;

Q – распределение вероятностей прогнозов модели.

Существуют разные подходы для нахождения минимального значения функции потерь, однако наиболее распространенным является метод градиентного спуска. Градиентный спуск – метод нахождения локального экстремума (минимума или максимума) функции с помощью движения вдоль градиента. Суть заключается в определении наклона графика функции с помощью частной производной. Наклон графика функции указывает на ближайшую впадину. На рисунке 3.1 отображен принцип нахождения минимума функции:

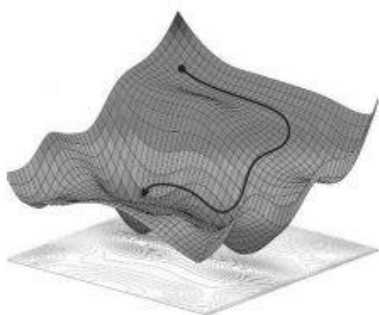
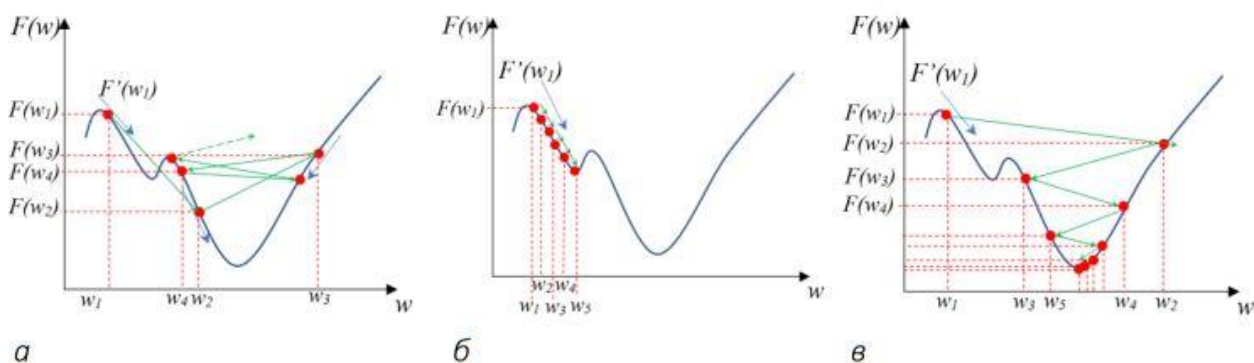


Рисунок 3.1. – Принцип нахождения минимума функции методом градиентного спуска

Метод градиентного спуска предполагает, что выбирается случайная точка, находится направление самого быстрого убывания функции и движется до ближайшего минимума вдоль этого направления (см. рисунок 3.1). Если функция в выбранной точке убывает, то вес следует увеличить, если функция возрастает – то уменьшить. Шаг этого изменения носит название *скорость обучения* и определяет, насколько будет сдвигаться точка на функции потерь.

Настройка скорости обучения так же очень важный процесс: если установить значение скорости обучения слишком большим, может быть вероятность того, что минимум вообще не будет достигнут, т. к. алгоритм всегда будет перешагивать его, а градиент разворачивать обратно (рисунок 3.2, а), если скорость будет очень малой величиной, то модель будет обучаться очень долго, и обучение может не завершиться, т. к. есть вероятность остановиться в локальном минимуме, где значение функции потерь будет неудовлетворительным (рисунок 3.2, б). Наиболее эффективным решением является подход, когда скорость обучения меняется по мере обучения (рисунок 3.2, в):



а – большая скорость обучения; **б** – низкая скорость обучения;
в – скорость обучения изменяется в процессе обучения
 Рисунок 3.2. – Метод градиентного спуска с разной скоростью обучения:
 $F(w)$ – значение функции потерь (величина ошибки),
 $F'(w)$ – частная производная в точке, w – вес

Пример обучения ИНС с использованием метода обратного распространения ошибки

Рассмотрим пример обучения ИНС, состоящей из скрытого и выходного слоя, каждый из которых включает три нейрона (рисунок 3.3) [17]. Начальные веса инициализируются случайным образом.

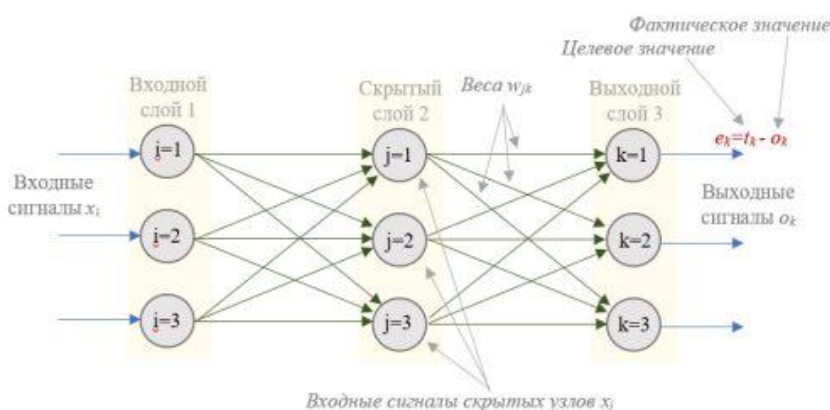


Рисунок 3.3. – Трехслойная полносвязная ИНС

Введем следующие обозначения: $w_{i,j}$ – веса между входным и скрытым слоями, $w_{j,k}$ – веса между скрытым и выходным слоями. Матрица весов представляется в виде

$$W = \begin{pmatrix} w_{1,1} & w_{2,1} & w_{3,1} \\ w_{1,2} & w_{2,2} & w_{3,2} \\ w_{1,3} & w_{2,3} & w_{3,3} \end{pmatrix}.$$

Допустим, на входе сети присутствуют данные $S = (0,9; 0,1; 0,8)^T$. Веса зададим случайным образом:

$$W_{\text{вх_скр}} = \begin{pmatrix} 0,9 & 0,3 & 0,4 \\ 0,2 & 0,8 & 0,2 \\ 0,1 & 0,5 & 0,6 \end{pmatrix}, \quad W_{\text{скр_вых}} = \begin{pmatrix} 0,3 & 0,7 & 0,5 \\ 0,6 & 0,5 & 0,2 \\ 0,8 & 0,1 & 0,9 \end{pmatrix}.$$

Метод обратного распространения ошибки предполагает, что сначала сигнал распространяется в прямом направлении. Входной сигнал для каждого нейрона на каждом слое

$$X = W \cdot S. \quad (3.1)$$

$$X_{\text{скр}} = \begin{pmatrix} 0,9 & 0,3 & 0,4 \\ 0,2 & 0,8 & 0,2 \\ 0,1 & 0,5 & 0,6 \end{pmatrix} \cdot \begin{pmatrix} 0,9 \\ 0,1 \\ 0,8 \end{pmatrix} = \begin{pmatrix} 1,16 \\ 0,42 \\ 0,62 \end{pmatrix}.$$

Таким образом, на вход скрытого слоя будут поступать значения 1,16; 0,42; 0,62.

К данным, поступающим на вход скрытого слоя, необходимо применить функцию активации, например, функцию сигмоиды $y = \frac{1}{1+e^{-x}}$. В результате на выходе скрытого слоя будут получены значения

$$Y_{\text{скр}} = \text{sigmoid} \begin{pmatrix} 1,16 \\ 0,42 \\ 0,62 \end{pmatrix} = \begin{pmatrix} \frac{1}{1+e^{-1,16}} \\ \frac{1}{1+e^{-0,42}} \\ \frac{1}{1+e^{-0,62}} \end{pmatrix} = \begin{pmatrix} 0,761 \\ 0,603 \\ 0,650 \end{pmatrix}.$$

По формуле (3.1) рассчитаем значения для выходного слоя:

$$X_{\text{ВЫХ}} = \begin{pmatrix} 0,3 & 0,7 & 0,5 \\ 0,6 & 0,5 & 0,2 \\ 0,8 & 0,1 & 0,9 \end{pmatrix} \cdot \begin{pmatrix} 0,761 \\ 0,603 \\ 0,650 \end{pmatrix} = \begin{pmatrix} 0,975 \\ 0,888 \\ 1,254 \end{pmatrix}.$$

После применения функции активации получаем значения на выходе нейронной сети (рисунок 3.4)

$$Y_{\text{ВЫХ}} = \text{sigmoid} \begin{pmatrix} 0,975 \\ 0,888 \\ 1,254 \end{pmatrix} = \begin{pmatrix} \frac{1}{1+e^{-0,975}} \\ \frac{1}{1+e^{-0,888}} \\ \frac{1}{1+e^{-1,254}} \end{pmatrix} = \begin{pmatrix} 0,726 \\ 0,708 \\ 0,778 \end{pmatrix}.$$

После применения функции активации получаем значения на выходе нейронной сети (см. рисунок 3.4)

$$Y_{\text{ВЫХ}} = \text{sigmoid} \begin{pmatrix} 0,975 \\ 0,888 \\ 1,254 \end{pmatrix} = \begin{pmatrix} \frac{1}{1+e^{-0,975}} \\ \frac{1}{1+e^{-0,888}} \\ \frac{1}{1+e^{-1,254}} \end{pmatrix} = \begin{pmatrix} 0,726 \\ 0,708 \\ 0,778 \end{pmatrix}.$$

Таким образом входные данные распространяются по сети в прямом направлении. Следующий шаг заключается в сравнении выходного сигнала с входными сигналами и вычислении ошибки с помощью функции потерь. Ошибка будет оказывать влияние на вес каждого входящего в нейрон сигнала пропорционально его вкладу.

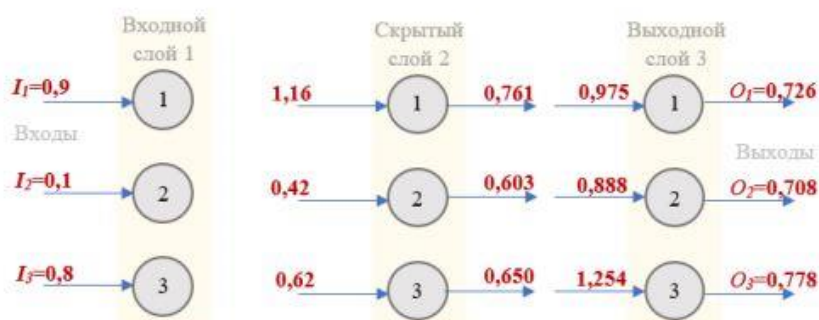


Рисунок 3.4. – Результат работы ИНС при случайной инициализации весов

В качестве функции потерь E будем использовать квадрат разности полученного результата и желаемого. Рассчитаем функцию потерь, которая представляет собой сумму возведенных в квадрат разностей между целевым и фактическим (выходным) значениями, где суммирование осуществлялось по всем n узлам:

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial}{\partial w_{jk}} \sum_n (t_n - y_n)^2.$$

Данное выражение можно упростить, исходя из того, что выходной сигнал y_n на узле n зависит лишь от весов w_{jk} , которые с ним соединены. Тогда

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial}{\partial w_{jk}} (t_k - y_k)^2.$$

В связи с тем, что целевое значение t_k – константа, то от w_{jk} зависит только фактическое значение y_k , и тогда

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial E}{\partial y_k} \cdot \frac{\partial y_k}{\partial w_{jk}} = -2(t_k - y_k) \cdot \frac{\partial y_k}{\partial w_{jk}},$$

где y_k – выходной сигнал узла предыдущего слоя, который, как рассматривалось выше, был получен в результате применения функции активации сигмоиды. Соответственно,

$$\frac{\partial E}{\partial w_{jk}} = -2(t_k - y_k) \cdot \frac{\partial}{\partial w_{jk}} \text{sigmoid}(\sum_j w_{jk} \cdot y_j).$$

Так как $\frac{\partial}{\partial x} \text{sigmoid}(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x))$,

$$\begin{aligned} \text{то } \frac{\partial E}{\partial w_{jk}} &= -2(t_k - y_k) \cdot \text{sigmoid}(\sum_j w_{jk} \cdot y_j)(1 - \text{sigmoid}(\sum_j w_{jk} \cdot y_j)) \cdot \frac{\partial}{\partial w_{jk}} (\sum_j w_{jk} \cdot y_j) = \\ &= -2(t_k - y_k) \cdot \text{sigmoid}(\sum_j w_{jk} \cdot y_j)(1 - \text{sigmoid}(\sum_j w_{jk} \cdot y_j)) \cdot y_j. \end{aligned}$$

В связи с тем, что интерес представляет только направление градиента, то скалярный множитель два можно игнорировать. Тогда окончательное выражение, которое будет использоваться для изменения веса w_{jk} ,

$$\frac{\partial E}{\partial w_{jk}} = -(t_k - y_k) \cdot \text{sigmoid}(\sum_j w_{jk} \cdot y_j) (1 - \text{sigmoid}(\sum_j w_{jk} \cdot y_j)) \cdot y_j. \quad (3.2)$$

Полученное выражение, по сути, является градиентом функции потерь. На его основе могут быть получены новые значения весовых коэффициентов w_{jk}^v :

$$w_{jk}^v = w_{jk}^{v-1} - \alpha \cdot \frac{\partial E}{\partial w_{jk}},$$

где w_{jk}^{v-1} – весовые коэффициенты предыдущего шага прямого прохода ИНС;

α – скорость обучения, знак «минус» перед поправкой позволяет увеличивать вес при отрицательной производной и уменьшать при положительной.

Аналогичным образом изменение весов выполняется на каждом слое. Одним из способов вычисления ошибки на скрытом слое ($e_{\text{скр}}$) является распределение ошибок выходного слоя ($e_{\text{вых}}$) между соответствующими связями пропорционально весу каждой связи с последующим объединением соответствующих разрозненных частей ошибки на каждом внутреннем узле:

$$e_{\text{скр}} = W_{\text{скр_вых}}^T \cdot e_{\text{вых}}, \quad (3.3)$$

где $W_{\text{скр_вых}}^T$ – транспонированная матрица весов между скрытым и выходным слоем.

Рассчитаем новые весовые коэффициенты между скрытым и выходным слоями. В выражении (3.2) величина ошибки ($t_k - y_k$) для выходного слоя будет равна

$$e_{\text{вых}} = \begin{pmatrix} 0,9 - 0,726 \\ 0,1 - 0,708 \\ 0,8 - 0,778 \end{pmatrix} = \begin{pmatrix} 0,174 \\ -0,608 \\ 0,022 \end{pmatrix}.$$

Второй множитель выражения (3.2) $\text{sigmoid}(\sum_j w_{jk} \cdot y_j)$ будет равен выходу сети:

$$\text{sigmoid} \begin{pmatrix} 0,3 & 0,7 & 0,5 \\ 0,6 & 0,5 & 0,2 \\ 0,8 & 0,1 & 0,9 \end{pmatrix} \cdot \begin{pmatrix} 0,761 \\ 0,603 \\ 0,650 \end{pmatrix} = \begin{pmatrix} \frac{1}{1+e^{-0,975}} \\ \frac{1}{1+e^{0,888}} \\ \frac{1}{1+e^{-1,254}} \end{pmatrix} = \begin{pmatrix} 0,726 \\ 0,708 \\ 0,778 \end{pmatrix}.$$

Третий множитель выражения (3.2) равен

$$1 - \text{sigmoid}(\sum_j w_{jk} \cdot y_j) = \begin{pmatrix} 0,274 \\ 0,292 \\ 0,222 \end{pmatrix}.$$

Перемножив соответствующие элементы матриц согласно выражению (3.2), получим

$$\frac{\partial E}{\partial w_{jk}} = \begin{pmatrix} -0,026 \\ 0,076 \\ -0,002 \end{pmatrix}.$$

Тогда новые весовые коэффициенты между скрытым и выходным слоями по выражению (3.3), если скорость обучения $\alpha = 0,5$

$$\begin{aligned} w_{jk}^v &= \begin{pmatrix} 0,3+0,5 \cdot 0,026 & 0,7+0,5 \cdot 0,026 & 0,5+0,5 \cdot 0,026 \\ 0,6-0,5 \cdot 0,076 & 0,5-0,5 \cdot 0,076 & 0,2-0,5 \cdot 0,076 \\ 0,8+0,5 \cdot 0,002 & 0,1+0,5 \cdot 0,002 & 0,9+0,5 \cdot 0,002 \end{pmatrix} = \\ &= \begin{pmatrix} 0,313 & 0,713 & 0,513 \\ 0,562 & 0,462 & 0,162 \\ 0,801 & 0,101 & 0,901 \end{pmatrix}. \end{aligned}$$

Рассчитаем новые весовые коэффициенты между входным и скрытым слоями. Ошибка на скрытом слое

$$e_{\text{скр}} = \begin{pmatrix} 0,3 & 0,7 & 0,5 \\ 0,6 & 0,5 & 0,2 \\ 0,8 & 0,1 & 0,9 \end{pmatrix}^T \cdot \begin{pmatrix} 0,174 \\ -0,608 \\ 0,022 \end{pmatrix} = \begin{pmatrix} 0,3 & 0,6 & 0,8 \\ 0,7 & 0,5 & 0,1 \\ 0,5 & 0,2 & 0,9 \end{pmatrix} \cdot \begin{pmatrix} 0,174 \\ -0,608 \\ 0,022 \end{pmatrix} = \begin{pmatrix} -0,295 \\ -0,18 \\ -0,0148 \end{pmatrix}.$$

Второй множитель выражения (3.2) $\text{sigmoid}(\sum_j w_{jk} \cdot y_j)$ будет равен выходу скрытого слоя:

$$\text{sigmoid} \begin{pmatrix} 0,9 & 0,3 & 0,4 \\ 0,2 & 0,8 & 0,2 \\ 0,1 & 0,5 & 0,6 \end{pmatrix} \cdot \begin{pmatrix} 0,9 \\ 0,1 \\ 0,8 \end{pmatrix} = \begin{pmatrix} \frac{1}{1+e^{-1,16}} \\ \frac{1}{1+e^{-0,42}} \\ \frac{1}{1+e^{-0,62}} \end{pmatrix} = \begin{pmatrix} 0,761 \\ 0,603 \\ 0,650 \end{pmatrix}.$$

Третий множитель выражения (3.2) равен

$$1 - \text{sigmoid}(\sum_j w_{jk} \cdot y_j) = \begin{pmatrix} 0,239 \\ 0,397 \\ 0,35 \end{pmatrix},$$

тогда

$$\frac{\partial E}{\partial w_{jk}} = \begin{pmatrix} 0,048 \\ 0,004 \\ 0,003 \end{pmatrix}.$$

И новые весовые коэффициенты между входным и скрытым слоями по выражению (3), если скорость обучения $\alpha = 0,5$

$$w_{jk}^v = \begin{pmatrix} 0,9 - 0,5 \cdot 0,048 & 0,3 - 0,5 \cdot 0,048 & 0,4 - 0,5 \cdot 0,048 \\ 0,2 - 0,5 \cdot 0,004 & 0,8 - 0,5 \cdot 0,004 & 0,2 - 0,5 \cdot 0,004 \\ 0,1 - 0,5 \cdot 0,003 & 0,5 - 0,5 \cdot 0,003 & 0,6 - 0,5 \cdot 0,003 \end{pmatrix} =$$

$$= \begin{pmatrix} 0,876 & 0,276 & 0,376 \\ 0,198 & 0,798 & 0,198 \\ 0,0985 & 0,4985 & 0,5985 \end{pmatrix}.$$

Как можно видеть, после одного прохода по сети веса изменяются довольно незначительно, однако после выполнения тысяч итераций весовые коэффициенты образуют устойчивую конфигурацию, которая позволит получать на выходе сети значения, соответствующие входным.

Выполним еще один прямой проход по сети. Входные значения скрытого слоя

$$X_{\text{скр}} = \begin{pmatrix} 0,876 & 0,276 & 0,376 \\ 0,198 & 0,798 & 0,198 \\ 0,0985 & 0,4985 & 0,5985 \end{pmatrix} \cdot \begin{pmatrix} 0,9 \\ 0,1 \\ 0,8 \end{pmatrix} = \begin{pmatrix} 0,1168 \\ 0,4164 \\ 0,6173 \end{pmatrix}.$$

После функции активации

$$Y_{\text{скр}} = \text{sigmoid} \begin{pmatrix} 0,1168 \\ 0,4164 \\ 0,6173 \end{pmatrix} = \begin{pmatrix} \frac{1}{1+e^{-1,1168}} \\ \frac{1}{1+e^{-0,4164}} \\ \frac{1}{1+e^{-0,6173}} \end{pmatrix} = \begin{pmatrix} 0,7534 \\ 0,6026 \\ 0,6496 \end{pmatrix}.$$

Для выходного слоя

$$X_{\text{вых}} = \begin{pmatrix} 0,313 & 0,713 & 0,513 \\ 0,562 & 0,462 & 0,162 \\ 0,801 & 0,101 & 0,901 \end{pmatrix} \cdot \begin{pmatrix} 0,7534 \\ 0,6026 \\ 0,6496 \end{pmatrix} = \begin{pmatrix} 0,9987 \\ 0,807 \\ 1,2496 \end{pmatrix}.$$

После применения функции активации получаем значения на выходе нейронной сети

$$Y_{\text{вых}} = \text{sigmoid} \begin{pmatrix} 0,9987 \\ 0,807 \\ 1,2496 \end{pmatrix} = \begin{pmatrix} \frac{1}{1+e^{-0,9987}} \\ \frac{1}{1+e^{-0,807}} \\ \frac{1}{1+e^{-1,2496}} \end{pmatrix} = \begin{pmatrix} 0,7308 \\ 0,6914 \\ 0,7772 \end{pmatrix}.$$

Если сравнить значения на выходе сети после первой и второй итерации, то можно заметить, что выходное значение первого нейрона немного увеличилось, а для второго нейрона немного уменьшилось, что говорит о верном направлении изменения весов и медленно приближает к целевому значению. Выходное значение третьего нейрона изначально было достаточно близким к целевому, и тут возможны колебания как в сторону увеличения

точности, так и в сторону небольшого снижения в процессе обучения, что произошло на текущей итерации, однако при дальнейшей корректировке весов на следующих итерациях это будет исправлено.

Рассмотренная в примере функция активации *sigmoid* нормализует значения нейронов в диапазоне от 0 до 1. В общем случае нормализация позволяет привести все выходные значения к одному масштабу и распределить их в диапазоне от $[-1, 1]$ или $[0, 1]$, что позволяет ускорить вычисления за счет того, что числа будут одного порядка. Кроме этого, ненормализованные данные приводят к неравномерному распределению весов, и часть из них будет иметь очень маленькие значения, а другая часть – очень большие, что будет приводить к тому, что прежде, чем алгоритм градиентного спуска сможет приблизиться к минимуму функции потерь, будут наблюдаться большие колебания в области плато.

Однако большинство функций активации не обладает свойством нормализации, и для того, чтобы привести все числовые значения к одному масштабу, между выходными значениями слоя и функцией активации вводится пакетная нормализация (batch normalization).

Суть пакетной нормализации заключается в следующем: пусть в функцию активации поступает вектор размерности d : $x = (x(1), \dots, x(d))$. Его нормализация по каждой из размерностей будет

$$\hat{x}^{(k)} = \frac{x^{(k)} - E(x^{(k)})}{\sqrt{D(x^{(k)})}},$$

где $E(x)$ – математическое ожидание;

$D(x)$ – дисперсия, которые вычисляются по всей обучающей выборке.

Применение пакетной нормализации позволяет получить нормальное распределение на каждом слое и ускорить обучение модели.

Для обучения предлагается использовать сверточную нейронную сеть YOLOv4-tiny, которая является упрощенной моделью YOLOv4, что позволит сократить временные затраты на обучение. Схема YOLOv4-tiny представлена на рисунке 3.5. YOLOv4-tiny также использует межэтапные соединения и остаточные блоки, однако в меньшем количестве. Другим отличием является обнаружение объектов только в двух масштабах ($N/32$ и $N/16$, где N – размер входного слоя) на 30-м и 37-м слое. В качестве функции активации после

сверточных слоев используется функция Leake-ReLU, функция потерь – CIOU (см. лабораторную работу № 2).

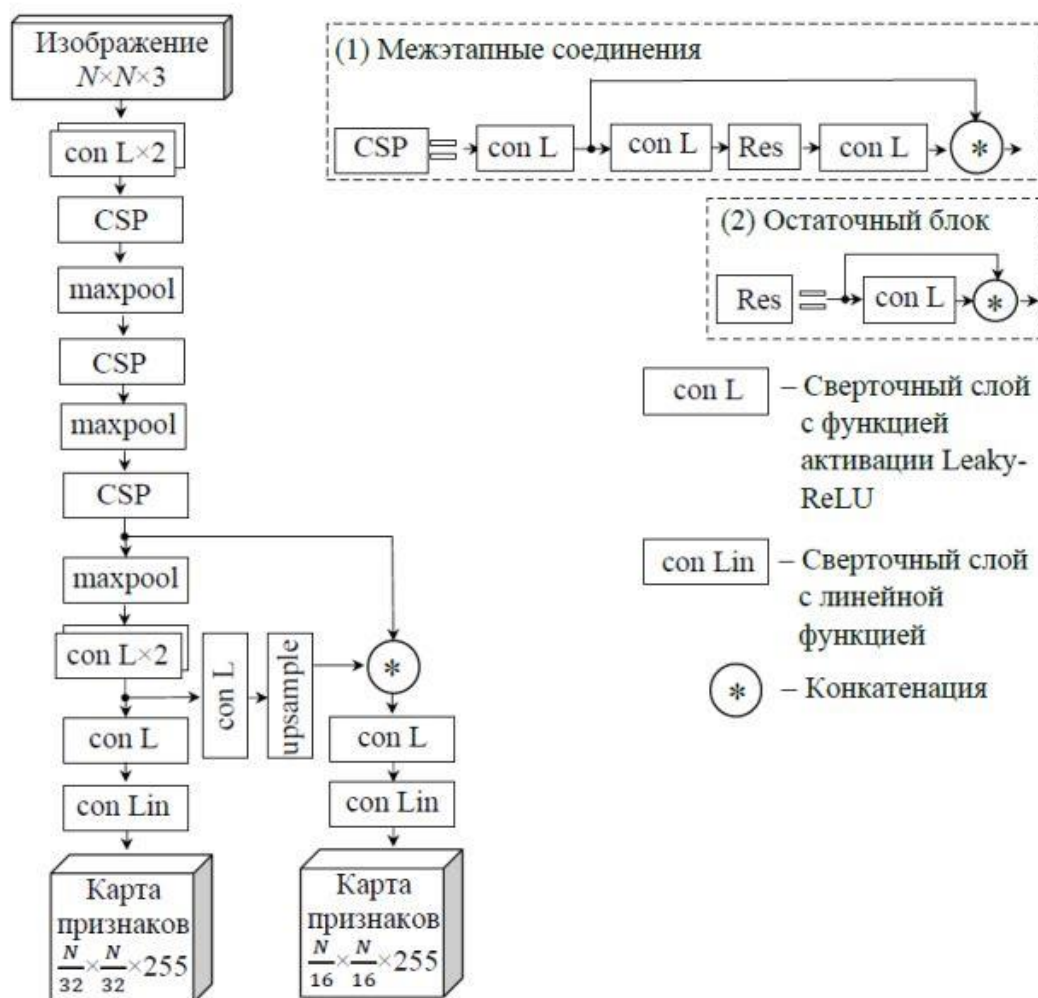


Рисунок 3.5. – Схема сверточной нейронной сети YOLOv4-tiny²

Практическая часть

Задание 1. Сформировать набор данных для обучения, включающий объекты согласно варианту заданий.

№ Варианта	Описание
Вариант 1	Бант, кастрюля, яблоко
Вариант 2	Гербера, собака, кружка
Вариант 3	Ноутбук, птица, клубника
Вариант 4	Клубника, вилка, кот
Вариант 5	Лошадь, велосипед, ромашка
Вариант 6	Гриб, ананас, кувшин
Вариант 7	Огурец, тарелка, гербера

Вариант 8	Кувшин, шляпа, лошадь
Вариант 9	Лилия, птица, земляника
Вариант 10	Роза, мотоцикл, огурец
Вариант 11	Улитка, яблоко, ведро
Вариант 12	Роза, карандаш, банан
Вариант 13	Бабочка, ежевика, велосипед
Вариант 14	Гриб, улитка, метла
Вариант 15	Лилия, бант, еж

Далее в качестве примера рассмотрим создание базы данных изображений для обучения сверточной нейронной сети YOLOv4 для детектирования объектов трех классов: **«подарок», «мышь» и «чайник»**. При формировании набора данных следует ориентироваться на следующие правила:

- 1- при разметке изображений убедитесь, что каждый объект правильно размечен и ни один не пропущен. Большинство проблем с обучением связано с некорректно размеченными данными для обучения;
- 2- для каждого объекта, который следует обнаруживать в обучающей выборке должен быть хотя бы один пример, содержащий похожий объект (по размеру, форме, цвету, углу поворота и т. д.);
- 3- набор данных должен включать изображения объектов с разных сторон, в разных масштабах и на разном фоне;
- 4- следует учитывать размер объектов на тренировочной и тестовой выборке: если в большинстве примеров объект занимает 80–90% изображения, то такой объект не будет обнаруживаться, если будет занимать 1–10% изображения.

Пример изображений для обучения показан на рисунке 1.

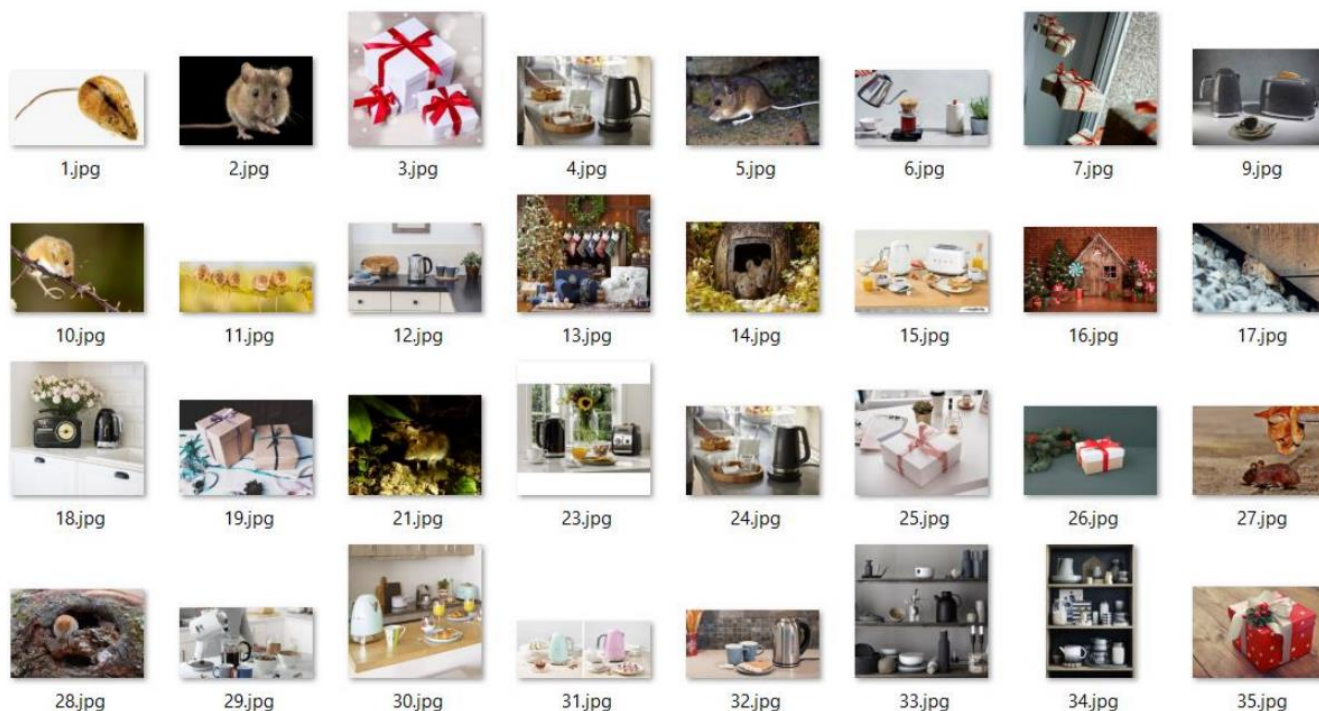


Рисунок 1 – Пример изображений для обучения

Разметка ограничительных рамок в ОС Windows

Репозиторий с графическим интерфейсом для разметки ограничительных рамок объектов и создания файлов аннотаций для YOLOv4: https://github.com/AlexeyAB/Yolo_mark

Клонирование репозитория на компьютер (рисунок 3.7):

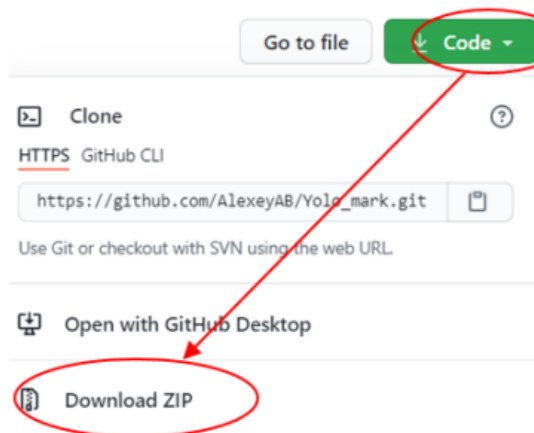


Рисунок 3.7. – Перенос репозитория на компьютер

Открыть yolo_mark.sln в MS Visual Studio. Далее перейти во вкладку «Проект» → «Свойства: yolo_mark.sln» → C/C++ → «Общие» → «Дополнительные каталоги включаемых файлов» и «Дополнительные каталоги» и указать путь к OpenCV, установленному компьютеру пользователя (рисунок 3.8). Затем выполнить компиляцию в x64&Release (рисунок 3.9).

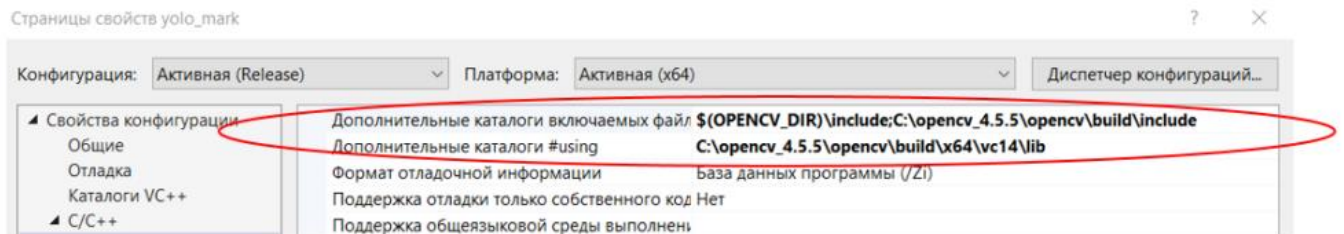


Рисунок 3.8. – Подключение OpenCV



Рисунок 3.9. – Выбор опций для компиляции в x64&Release

Для маркировки изображений необходимо выполнить действия:

- 1- удалить все файлы из каталога x64/Release/data/img;
- 2- поместить свои изображения .jpg в каталог x64/Release/data/img;
- 3- изменить количество классов объектов в файле x64/Release/data/obj.data, указав число 3;

```
classes= 3
train  = data/train.txt
valid  = data/train.txt
names  = data/obj.names
backup = backup/
```

- 4- редактировать файл x64/Release/data/obj.names в соответствии с именами классов изображений (рисунок 3.10);

obj.names – Блокнот

Файл Правка Формат
mouse
present_box
kettle

Рисунок 3.10. – Отображение содержимого файла obj.names

- 5- запустить файл x64/Release/yolo_mark.cmd;
- 6- для каждого изображения необходимо выбрать class id и поместить в прямоугольник соответствующий объект, после чего перейти к следующему изображению. Пример разметки объекта показан на рисунок 3.11.

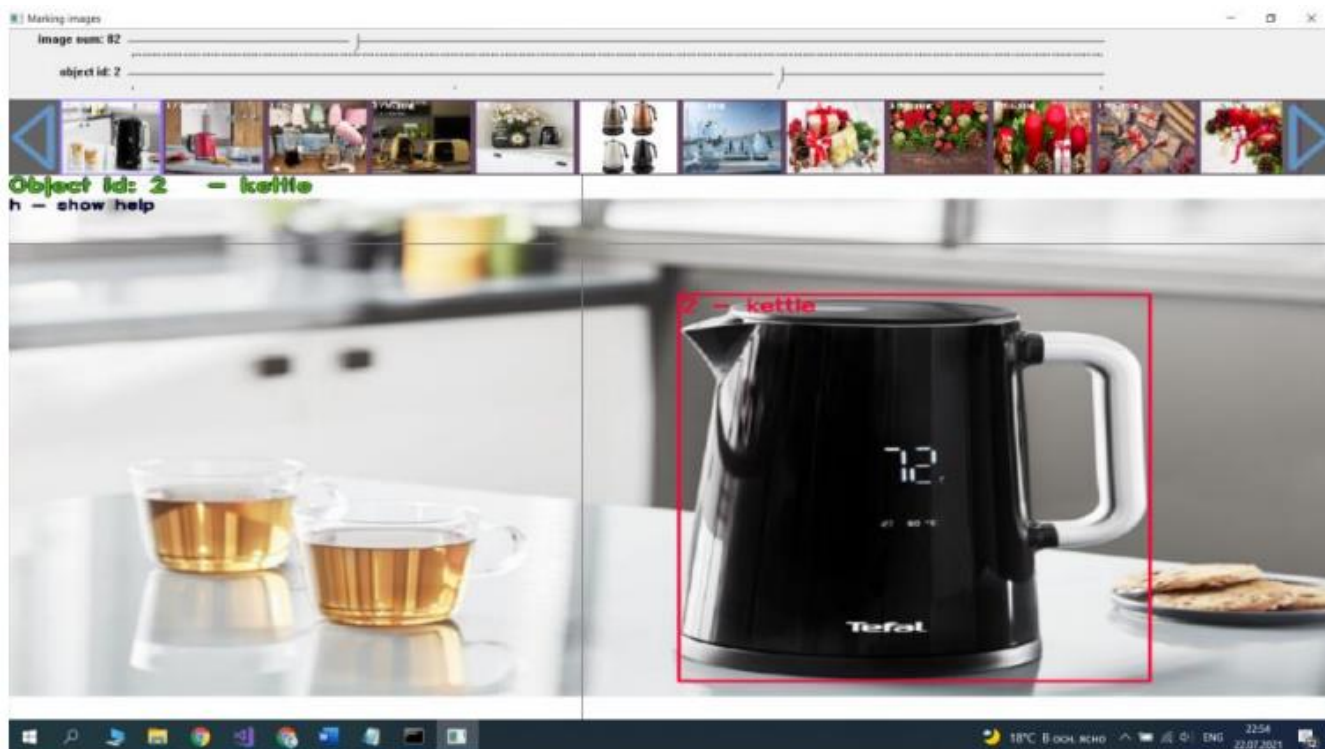


Рисунок 3.11. – Разметка объекта класса «чайник»

В результате работы программы для каждого размеченного изображения будет создан отдельный .txt файл с координатами выбранных объектов, и один файл train.txt, содержащий адреса расположения всех рассмотренных изображений.

Таблица 3.3. – Назначение клавиш клавиатуры для разметки изображений

Клавиша	Действие
	Следующее изображение
	Предыдущее изображение
	Удалить выделенное поле (при наведении курсора)
	Удалить все отметки на текущем изображении
	Копировать предыдущую отметку
	Следить за объектами
	Закреть приложение
	Один объект на изображении
	Идентификатор объекта
	Показать координаты
	Ширина линии
	Скрыть название объекта
	Помощь

Обучение CHC YOLOv4-tiny

Для возможности запуска CHC YOLO необходимо установить фреймворк Darknet одним из рассмотренных ниже способов. Для этого нужно загрузить архив Darknet с официального репозитория (<https://github.com/AlexeyAB/darknet>) и распаковать его.

Одним из способов компиляции Darknet является использование утилиты CMake. В CMake заполняется поле адреса расположения фреймворка Darknet и параметры для сборки проекта. На рисунке 3.12 цифрами обозначена последовательность действий.

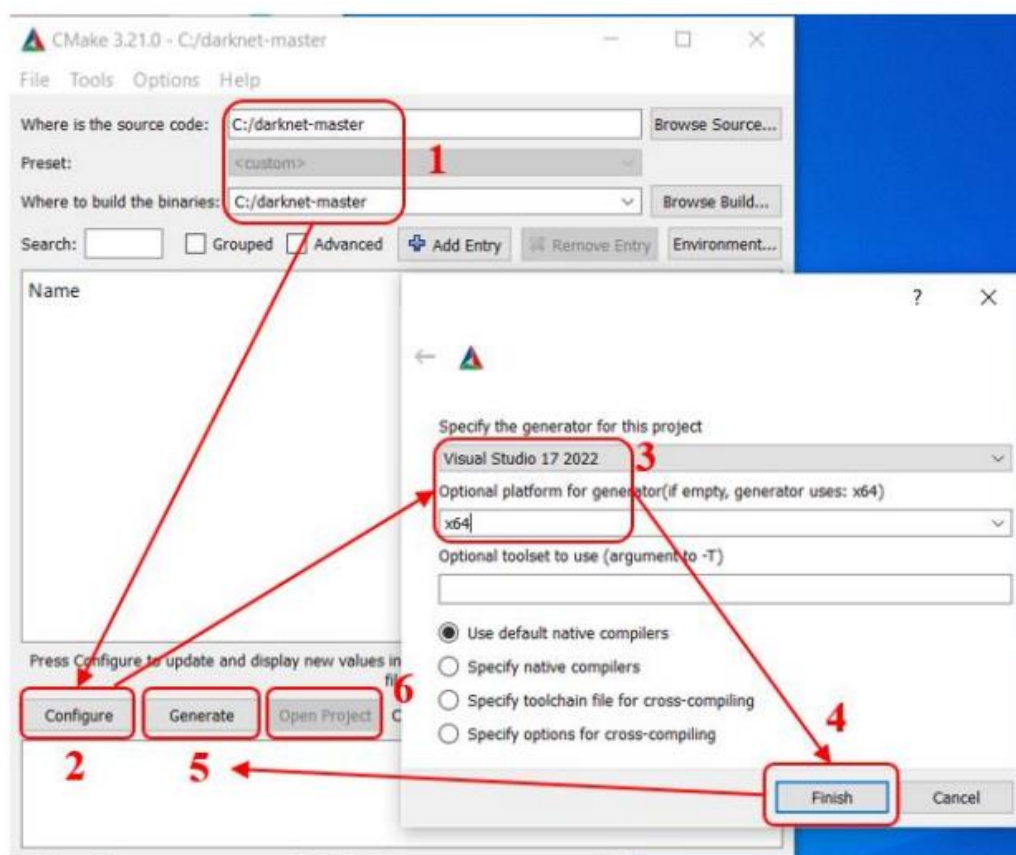


Рисунок 3.12. – Настройки для в CMake для компиляции фреймворка darknet

После нажатия Open Project откроется окно MS Visual Studio, где необходимо выбрать Release и x64, после чего выполнить сборку (рисунок 3.13).

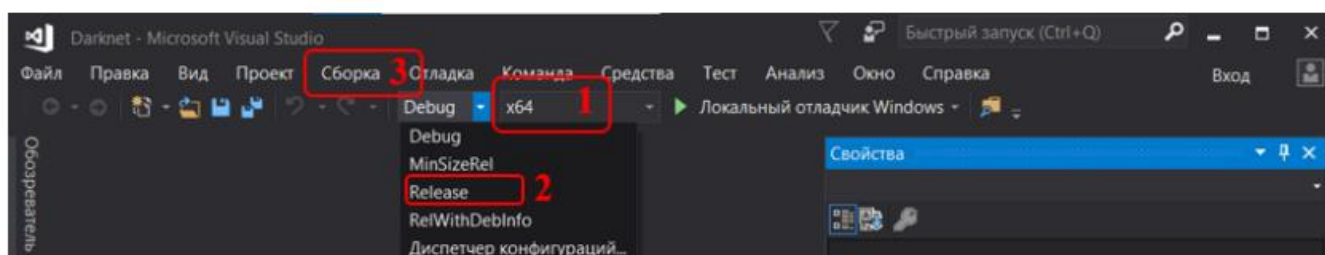


Рисунок 3.13. – Порядок действий в MS Visual Studio

При наличии видеокарты Nvidia с поддержкой CUDA можно использовать следующий способ компиляции Darknet:

1- установить MS Visual Studio;

2- установить CUDA, включив интеграцию с MSVS во время установки. Пояснения по установке CUDA и cuDNN приведены в приложении 1;

3- скачать и распаковать Darknet;

4- открыть с помощью приложения «Блокнот» файл build/darknet/darknet.vcxproj;

5- заменить следующие строки на значения, соответствующие пользовательским установкам и параметрам компьютера пользователя:

- C:/opencv_2.4.9/opencv/... на C:/opencv_4.5.5/opencv/... (или другую версию OpenCV, установленную на компьютере пользователя);

- CUDA 11.1.props ... на CUDA 11.4.props (или другую версию CUDA, установленную на компьютере пользователя);

- CUDA 11.1.targets ... на CUDA 11.4.targets (или другую версию CUDA, установленную на компьютере пользователя);

- compute_30, sm30 на compute_86, sm86 (проверить подходящее для пользователя значение можно по ссылке <http://arnon.dk/matching-sm-architectures-arch-and-gencode-for-various-nvidia-cards/>).

6- сохранить изменения;

7- запустить Build/darknet/darknet.sln с помощью MS VS;

8- в настройки проекта добавить:

- Include path:

C:\opencv_4.5.5\opencv\build\include;

C:\opencv_4.5.5\opencv\sources\3rdparty\include;

\$(CUDA_PATH)\include;

- Library path:

C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.4\lib\x64;

\$(CUDA_PATH)\lib\\$(PlatformName);

C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v8.0\lib\x64;

- Linker input:

..\..\3rdparty\lib\x64\pthreadVC2.lib;cublas.lib;curand.lib;cudaart.lib;

9- выполнить сборку (см. рисунок 3.13).

Возможно возникновение ошибки, как показано на рисунке 3.14.

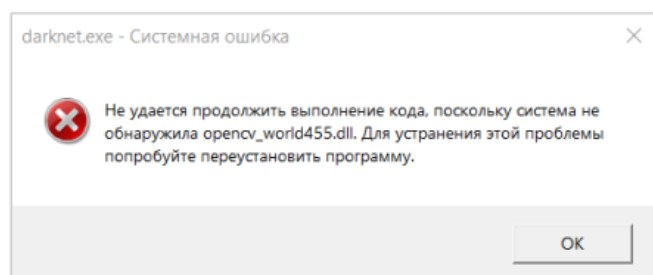


Рисунок 3.14. – Отображение системной ошибки при сборке фреймворка darknet

При такой ошибке следует скопировать файлы «opencv_world455.dll» и «opencv_world455d.dll» из «C:\opencv_4.5.5\opencv\build\x64\vc14\bin» в «C:\Windows\System32». Другие способы установки рассмотрены на вебресурсе <https://github.com/AlexeyAB/darknet>.

Для обучения *yolov4-tiny.cfg* необходимо выполнить следующую последовательность действий:

- 1- создать в каталоге /darknet-master/build/darknet/x64 файл yolov4-tiny-obj.cfg с тем же содержимым, что и файл yolov4-tiny.cfg;
- 2- для обучения СНС обнаружению заданного количества классов файл .cfg необходимо отредактировать:

- изменить размер пакета: `batch = 64` и `subdivision = 16`;
- изменить строку `max_batches` на `classes*2000`, но не менее количества обучающих образцов и не менее 6000. Например, если количество классов = 3, то `max_batches = 6000`;
- изменить строку `step` на значение, составляющее 80% и 90% от `max_batches`, т. е. `step = 4800, 5400`;
- размер входного слоя: `weight = 416, height = 416` или любое значение, кратное 32.
- измените строку `classes = 80` на ваше значение числа классов для каждого из двух слоев обнаружения `yolo`;
- для каждого из двух слоев, предшествующих слою `yolo`, необходимо изменить количество фильтров с `filters = 255` на значение, равное $(classes + 5) \times 3$. Для трех классов значение `filters = 24`;

- 3- скачать файл предобученных весов для сверточных слоев и поместить их в каталог darknet-master/build/darknet/x64:

- для СНС yolov4-tiny.cfg:
https://github.com/AlexeyAB/darknet/releases/download/darknet_yolo_v4_pre/yolov4-tiny.conv.29

- 4- созданные ранее файлы `obj.data` и `obj.names` скопировать в каталог /darknet-master/build/darknet/x64/data;

5- созданный ранее набор данных переместить в каталог /darknetmaster/build/darknet/x64/data/img вместе с .txt-файлами, созданными при разметке;

6- созданный при разметке файл train.txt переместить в каталог /darknet-master/build/darknet/x64/data;

7- начать обучение командной строки (рисунок 3.15):

```
darknet.exe detector train data/obj.data yolov4-tiny-obj.cfg yolov4-tiny.conv.29 -gpus 0
```

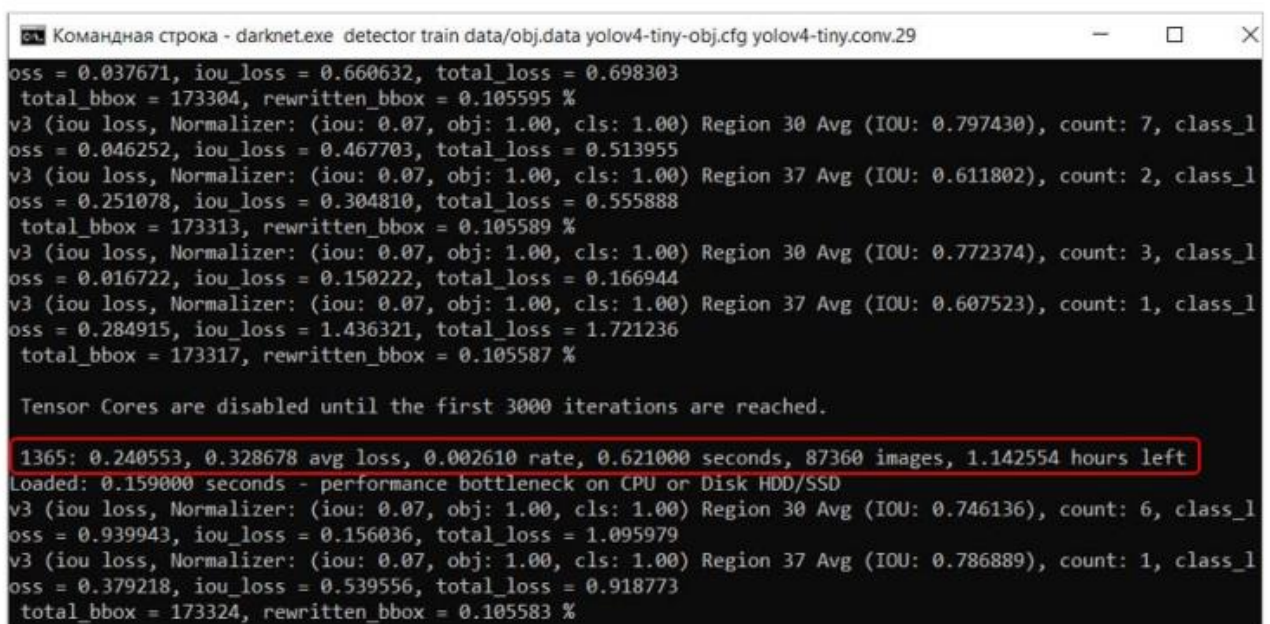
- gpus 0 обеспечивает обучение на GPU. Если используется более одной видеокарты, то первую 1000 итераций обучения необходимо выполнить на первой GPU, затем остановить обучение и продолжить его на уже частично обученной модели с использованием нескольких видеокарт: darknet.exe detector train data/obj.data yolov4-tiny-obj.cfg yolov4-tiny.conv.29 -gpus 0,1,2,3

```
C:\darknet-master\build\darknet\x64>darknet.exe detector train data/obj.data yolov4-tiny-obj.cfg yolov4-tiny.conv.29 -gpus 0
```

Рисунок 3.15. – Командная строка для начала обучения

При обучении на CPU «-gpus 0» не указывать;

8- если возникает ошибка нехватки памяти, то следует уменьшить размер пакета или увеличить Subdivision до 32 или 64, но не более, чем размер пакета. Пример окна командной строки при обучении показан на рисунке 3.16.



```
Командная строка - darknet.exe detector train data/obj.data yolov4-tiny-obj.cfg yolov4-tiny.conv.29
oss = 0.037671, iou_loss = 0.660632, total_loss = 0.698303
total_bbox = 173304, rewritten_bbox = 0.105595 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.797430), count: 7, class_1
oss = 0.046252, iou_loss = 0.467703, total_loss = 0.513955
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.611802), count: 2, class_1
oss = 0.251078, iou_loss = 0.304810, total_loss = 0.555888
total_bbox = 173313, rewritten_bbox = 0.105589 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.772374), count: 3, class_1
oss = 0.016722, iou_loss = 0.150222, total_loss = 0.166944
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.607523), count: 1, class_1
oss = 0.284915, iou_loss = 1.436321, total_loss = 1.721236
total_bbox = 173317, rewritten_bbox = 0.105587 %

Tensor Cores are disabled until the first 3000 iterations are reached.

1365: 0.240553, 0.328678 avg loss, 0.002610 rate, 0.621000 seconds, 87360 images, 1.142554 hours left
Loaded: 0.159000 seconds - performance bottleneck on CPU or Disk HDD/SSD
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.746136), count: 6, class_1
oss = 0.939943, iou_loss = 0.156036, total_loss = 1.095979
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.786889), count: 1, class_1
oss = 0.379218, iou_loss = 0.539556, total_loss = 0.918773
total_bbox = 173324, rewritten_bbox = 0.105583 %
```

Рисунок 3.16. – Командная строка при обучении: 1365 – текущая итерация / пакет обучения; 0.240553 – общая потеря; 0.328678 – среднее значение потерь (avg loss); 0.002610 – текущая скорость обучения; 0.621000 – общее время, затраченное на обработку пакета; 87360 – общее количество изображений, обработанное к текущему моменту

Среднее значение потерь, должно быть как можно меньше. Хорошим результатом считается, если это значение меньше чем 0,05. Однако при сложном наборе данных и большой модели это значение может достигать значения 3.

Кроме этого, отображается .png-файл, отражающий график потерь при обучении. Пример графика по окончании обучения показан на рисунке 3.17. Для итерации с номером 5864 получено среднее значение потерь $\text{avg loss} = 0,0661$, что является удовлетворительным результатом.

Другие параметры, с которыми было завершено обучение для данного примера, отражены в командной строке (рисунок 3.18). Из рисунка видно, что среднее значение потерь уменьшилось до $\text{avg loss} = 0,063014$.

Для обнаружения объектов заданных классов с помощью обученной СНС можно использовать программный код из лабораторной работы № 2 или применить командную строку (рисунок 3.19).

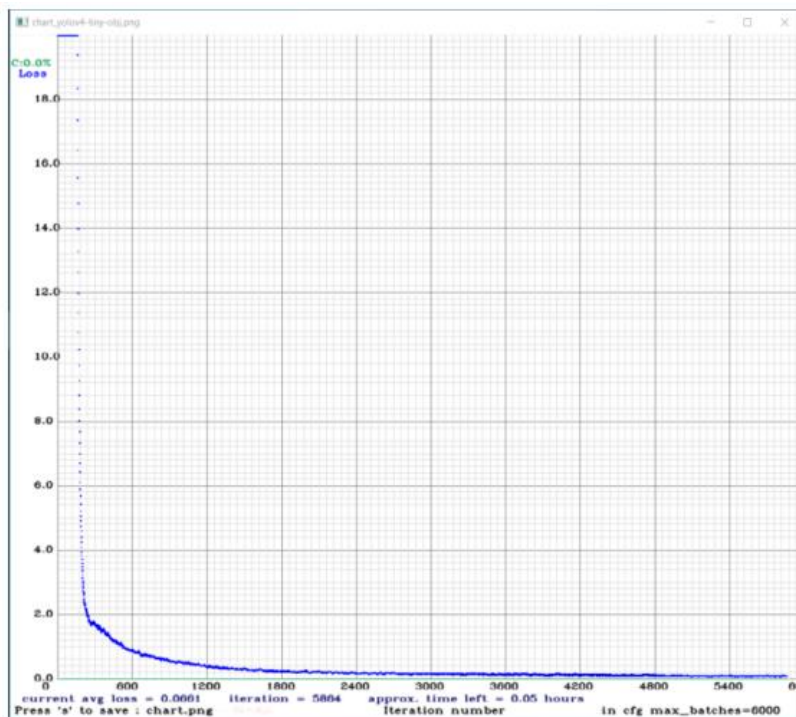


Рисунок 3.17. – График потерь при обучении СНС

```
6000: 0.055966, 0.063014 avg loss, 0.000026 rate, 0.607000 seconds, 384000 images, 0.022319 hours left
Saving weights to backup//yolov4-tiny-obj_6000.weights
Saving weights to backup//yolov4-tiny-obj_last.weights
Saving weights to backup//yolov4-tiny-obj_final.weights
If you want to train from the beginning, then use flag in the end of training command: -clear
```

Рисунок 3.18. – Отображение достигнутых параметров обучения СНС в командной строке

```
C:\darknet-master\build\darknet\x64>darknet.exe detector test data/obj.data yolov4-tiny-obj.cfg backup-tiny/yolov4-tiny-obj_final.weights data/3.jpg
```

Рисунок 3.19. – Командная строка для обнаружения объектов на изображении с использованием обученной СНС

Для тестирования работы обученной СНС следует выбирать изображения, которые не входили в обучающую выборку.

Обучение сверточной нейронной сети YOLOv4-tiny с помощью облачного сервиса Google Colab

Google Colab – бесплатная интерактивная облачная среда для работы с кодом, которая предоставляет мощные графические процессоры GPU, а также тензорные процессоры TPU, позволяющие проводить сложные вычисления, в том числе и в области машинного обучения. Для работы в виртуальной среде Google Colab необходимо иметь учетную запись Google Cloud Disk: <https://drive.google.com>

При работе с Google Colab следует помнить, что после перезапуска среды загруженные файлы могут быть удалены.

Обучение СНС YOLOv4-tiny с использованием облачного сервиса Google Colab требует выполнения следующих шагов:

1- открыть Google Cloud Disk, затем с помощью правой кнопки мыши открыть всплывающее окно и выбрать функцию «Подключить другие приложения». В появившемся окне выбрать опцию «Colaboratory» (рисунок 3.20), выполнить установку и переход на данный продукт;

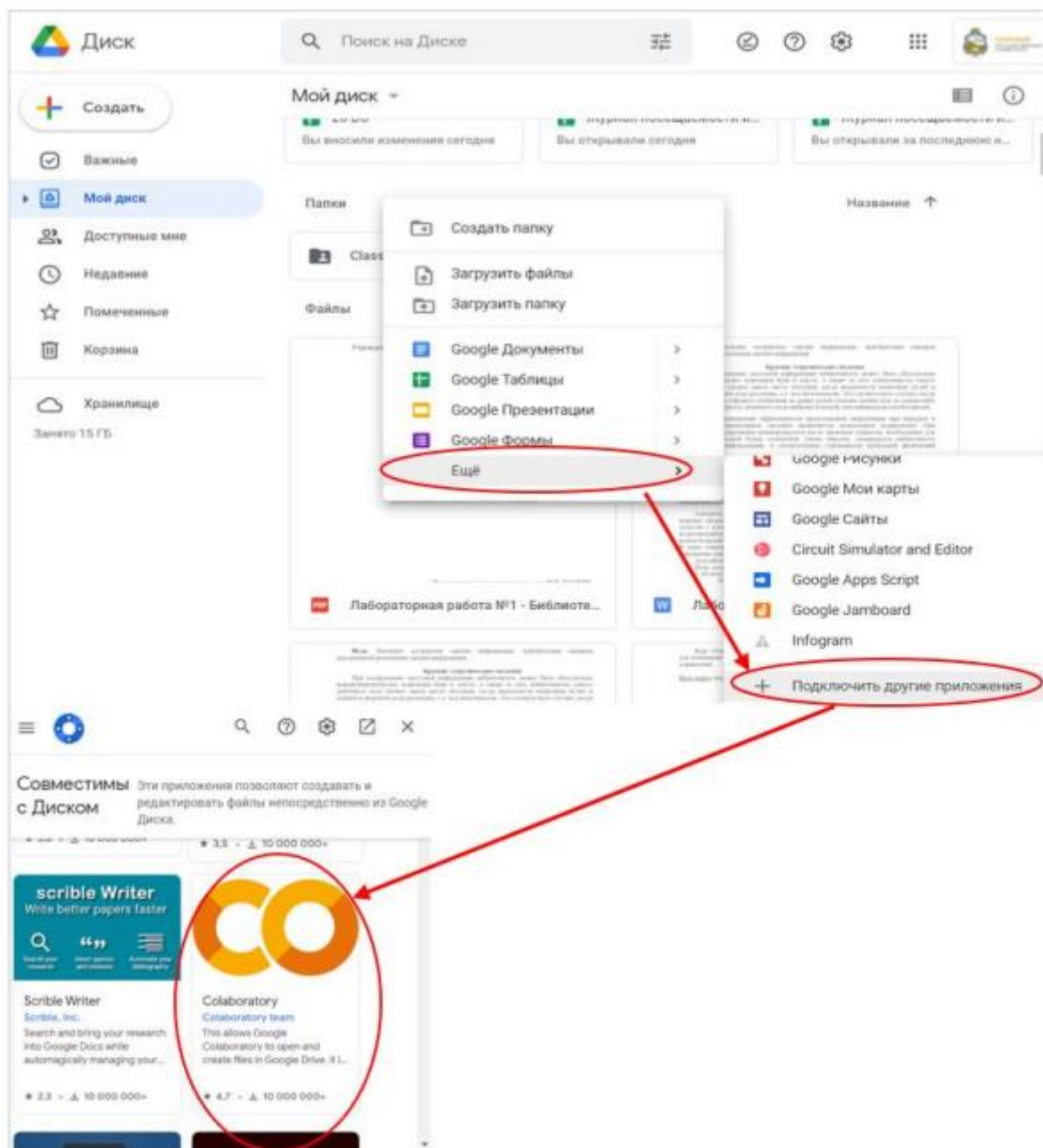


Рисунок 3.20. – Подключение бесплатной интерактивной облачной среды Colab

2- после перехода в Colaboratory на предыдущем шаге автоматически будет создан файл Untitled0.ipynb. Чтобы упростить передачу файлов с облачным графическим процессором, выделенным Colab, необходимо установить связь с Google Cloud Disk. Для этого в окне командной строки Untitled0.ipynb применить команды, как показано на рисунок 3.21. После успешного выполнения импорта данных станет доступен «My Drive» – файлы на Google-диске в Colab (рисунок 3.22);

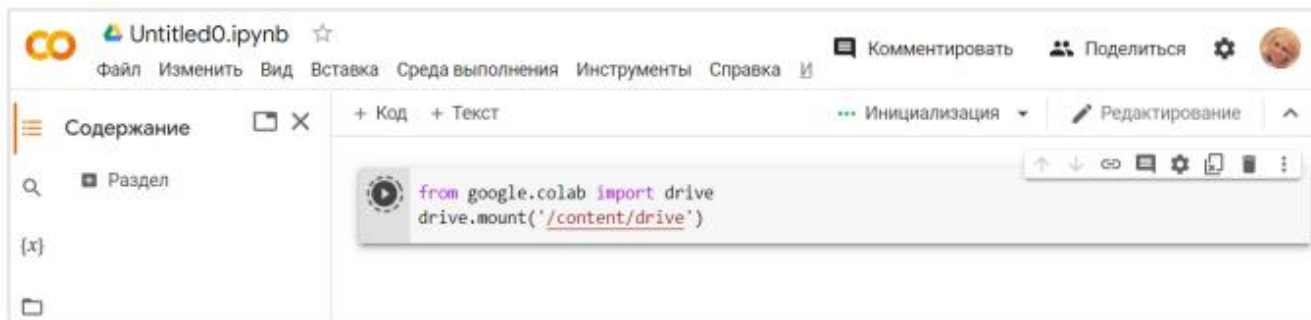
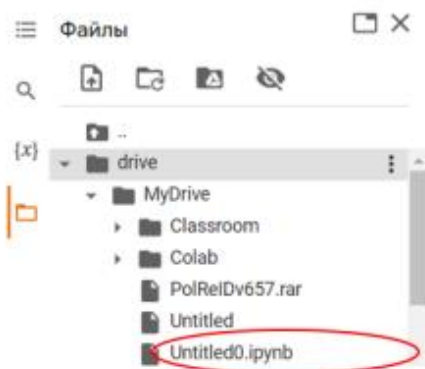


Рисунок 3.21. – Команды для импорта данных с Colab на Google Cloud Disk



**Рисунок 3.22. – Отображение файла
Untitled0.ipynb на Google Cloud Disk**

- 3- для удобства можно переименовать файл Untitled0.ipynb в YOLOv4.ipynb и переместить его в папку Colab;
- 4- выполнить действия по подключению графического процессора и проекта darknet с помощью git для возможности обучения в Colab:
- подключение графического процессора показано на рисунке 3.23;

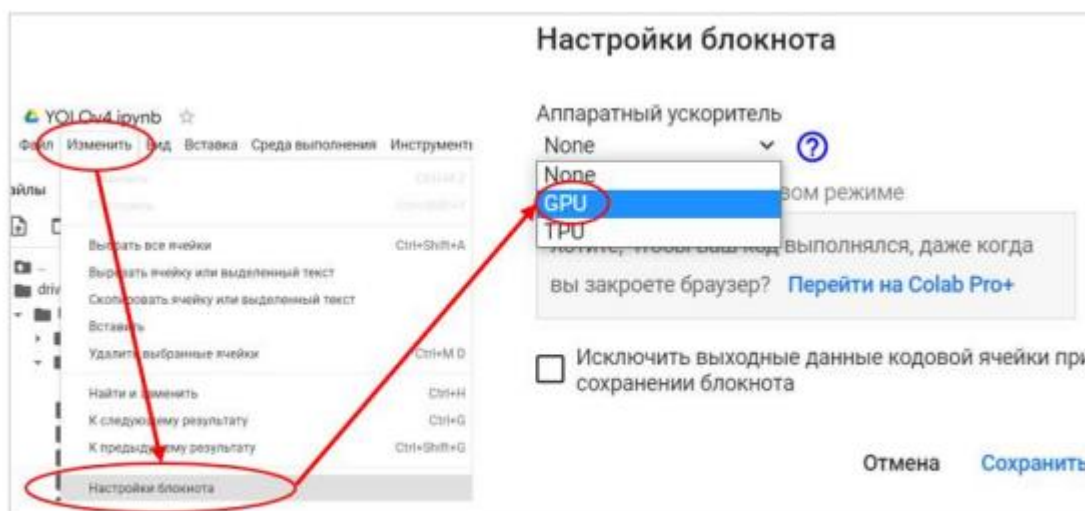


Рисунок 3.23. – Настройки для подключения графического процессора

- клонирование проекта для создания каталога darknet (рисунок 3.24). После успешного выполнения в окне «Файлы» будет создан каталог Darknet (рисунок 3.25). <https://colab.research.google.com/drive/>

```
!git clone https://github.com/AlexeyAB/darknet
```

Рисунок 3.24. – Клонирование проекта для создания каталога darknet

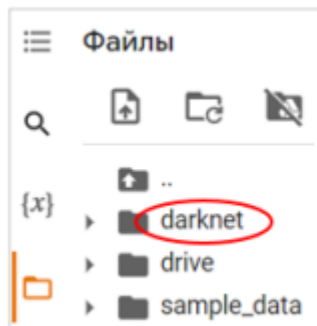
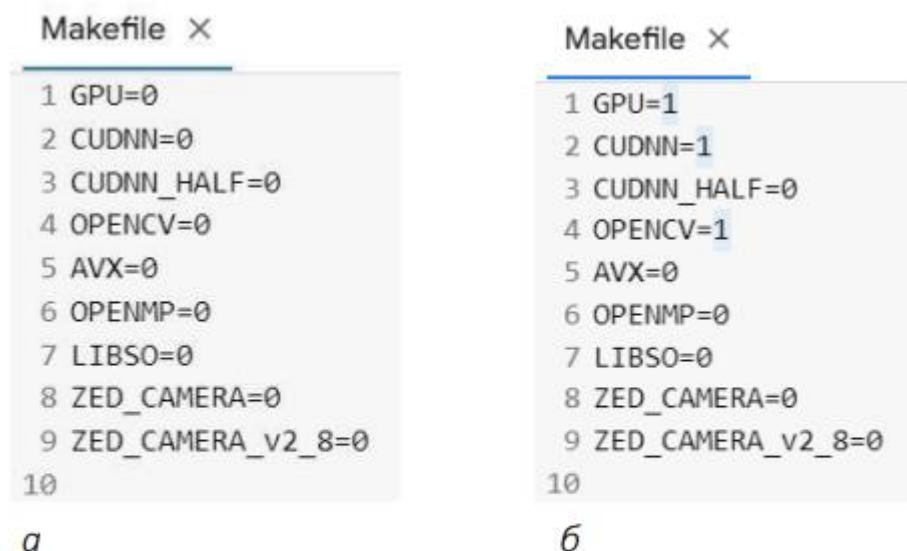


Рисунок 3.25. – Отображение каталога darknet на Google-диске

– изменить make-файл проекта с целью предоставления возможности скомпилированной модели CHC использования GPU и OpenCV:

```
%cd darknet
!sed -i 's/OPENCV=0/OPENCV=1/' Makefile
!sed -i 's/GPU=0/GPU=1/' Makefile
!sed -i 's/CUDNN=0/CUDNN=1/' Makefile
```

В случае успешного применения изменений содержимое Darknet/Makefile изменится для выбранных позиций (рисунок 3.26).



***a* – до изменений; *б* – после внесения изменений**

Рисунок 3.26. – Содержимое Makefile

1- скомпилировать проект Darknet командой !make;
2- в каталоге /darknet/cfg создать файл yolov4-tiny-obj.cfg путем переноса содержимого из файла yolov4-tiny.cfg и отредактировать его для обнаружения заданного количества классов:

а) изменить размер пакета: batch = 64 и subdivision = 16;

б) изменить строку `max_batches` на `classes*2000`, но не менее количества обучающих образцов и не менее 6000. Например, если количество классов = 3, то `max_batches` = 6000;

в) изменить строку `step` на значение, составляющее 80% и 90% от `max_batches`, т. е. `step` = 4800, 5400;

г) размер входного слоя: `weight` = 416, `height` = 416 или любое значение, кратное 32;

д) изменить строку `classes` = 80 на ваше значение числа классов для каждого из двух слоев обнаружения `yolo`;

е) для каждого из двух слоев, предшествующих слою `yolo`, необходимо изменить количество фильтров с `filters` = 255 на значение, равное $(classes + 5) \times 3$. Для трех классов значение `filters` = 24;

3- поместить в каталог `/darknet` файл предобученных весов для `yolov4-tiny.cfg`:

https://github.com/AlexeyAB/darknet/releases/download/darknet_yolo_v4_pre/yolov4-tiny.conv.29

4- созданные ранее файлы `obj.data`, `obj.names` и `train.txt` скопировать в каталог `/darknet/data`;

5- набор данных с файлами разметки `.txt` поместить в `/darknet/data/img`;

6- для старта обучения необходимо выполнить:

```
!./darknet detector train data/obj.data cfg/yolov4-tiny-obj.cfg yolov4-tiny.conv.29 -dont_show
```

При обучении отображается информация, показанная на рисунке 3.27.

```
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.173389), count: 4, class_loss = 1.811144, iou_loss = 0.030358, total_loss = 1.841502, rewritten_bbox = 0.107046 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.000000), count: 1, class_loss = 0.030358, iou_loss = 0.000000, total_loss = 0.030358, rewritten_bbox = 0.107046 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.372627), count: 5, class_loss = 2.625066, iou_loss = 0.110097, total_loss = 2.735163, rewritten_bbox = 0.107026 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.284107), count: 2, class_loss = 0.742174, iou_loss = 0.019336, total_loss = 0.761510, rewritten_bbox = 0.107026 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.217873), count: 3, class_loss = 1.350515, iou_loss = 0.012092, total_loss = 1.362607, rewritten_bbox = 0.107015 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.236072), count: 1, class_loss = 0.583221, iou_loss = 0.019295, total_loss = 0.602516, rewritten_bbox = 0.107015 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.157263), count: 6, class_loss = 2.739183, iou_loss = 0.010528, total_loss = 2.749711, rewritten_bbox = 0.106992 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.497920), count: 2, class_loss = 0.905029, iou_loss = 0.196792, total_loss = 1.101821, rewritten_bbox = 0.106992 %
296: 1.483794, 1.621493 avg loss, 0.000020 rate, 0.867034 seconds, 18944 images, 2.816685 hours left
Loaded: 0.490785 seconds - performance bottleneck on CPU or Disk HDD/SSD
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.226634), count: 5, class_loss = 1.911020, iou_loss = 0.017343, total_loss = 1.928363, rewritten_bbox = 0.106978 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.000000), count: 1, class_loss = 0.030358, iou_loss = 0.000000, total_loss = 0.030358, rewritten_bbox = 0.106978 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.305313), count: 9, class_loss = 4.123933, iou_loss = 0.175356, total_loss = 4.299289, rewritten_bbox = 0.106935 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.369110), count: 6, class_loss = 2.906950, iou_loss = 0.292388, total_loss = 3.209338, rewritten_bbox = 0.106935 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.402281), count: 11, class_loss = 4.694819, iou_loss = 0.308326, total_loss = 5.003145, rewritten_bbox = 0.106880 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.294907), count: 8, class_loss = 3.226195, iou_loss = 0.384096, total_loss = 3.610291, rewritten_bbox = 0.106880 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.252941), count: 5, class_loss = 2.479150, iou_loss = 0.038982, total_loss = 2.518132, rewritten_bbox = 0.106863 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 37 Avg (IOU: 0.360814), count: 1, class_loss = 0.436258, iou_loss = 0.035234, total_loss = 0.471492, rewritten_bbox = 0.106863 %
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 30 Avg (IOU: 0.364564), count: 4, class_loss = 1.378708, iou_loss = 0.027701, total_loss = 1.406409, rewritten_bbox = 0.106863 %
```

Рисунок 3.27. – Отображение данных при обучении СНС на облачном сервисе Google Colab: 296 – текущая итерация / пакет обучения; 1.483794 – общие потери; 1.621493 – среднее значение потерь; 0.000020 – текущая скорость обучения; 0.867034 – общее время, затраченное на обработку пакета; 18974 – общее количество изображений, обработанное к текущему моменту; 2,816685 – приблизительное значение времени, которое необходимо для завершения обучения СНС

Отличием обучения на облачном сервисе Google Colab от обучения на GPU компьютера является отсутствие возможности просмотра графика потерь в режиме онлайн, однако его можно загрузить двойным щелчком мыши или скачать. Также он будет доступен по окончании тренировки СНС (рисунок 3.28).

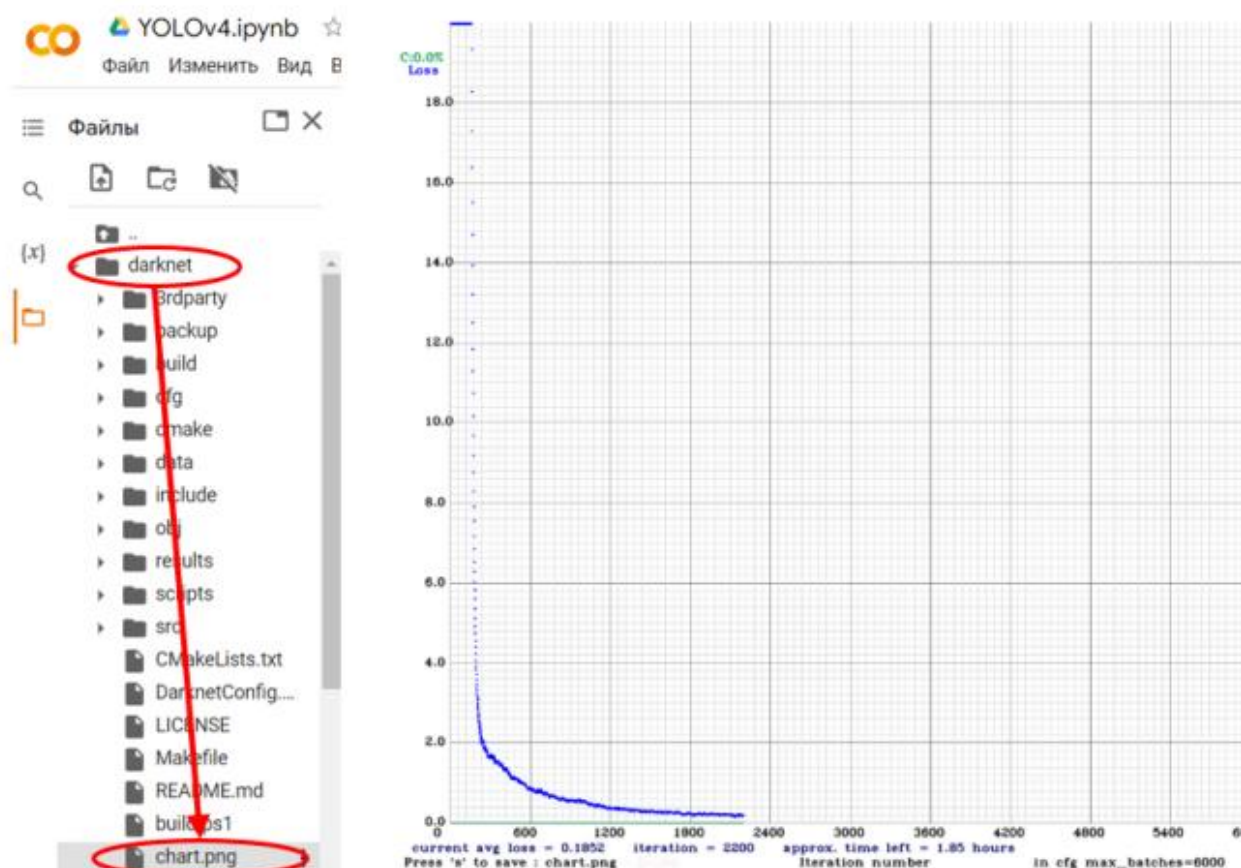


Рисунок 3.28. – График потерь при обучении СНС на облачном сервисе Google Colab

По окончании обучения отображаются все данные и сообщение о сохранении весов СНС (рисунок 3.29).

```
6000: 0.057582, 0.062380 avg loss, 0.000026 rate, 0.902357 seconds, 384000 images, 0.047547 hours left
Saving weights to backup//yolov4-tiny-obj_6000.weights
Saving weights to backup//yolov4-tiny-obj_last.weights
Saving weights to backup//yolov4-tiny-obj_final.weights
If you want to train from the beginning, then use flag in the end of training command: -clear
```

Рисунок 3.29. – Результаты обучения СНС на облачном сервисе Google Colab

Для применения обученной СНС к задаче обнаружения объектов заданных классов необходимо использовать командную строку:

```
!./darknet detector test data/obj.data cfg/yolov4-tiny-obj.cfg yolov4-tiny-obj_final.weights data/3.jpg
```

В результате выполнения обнаружения объектов заданных классов создается файл Prediction.jpg (рисунок 3.30)

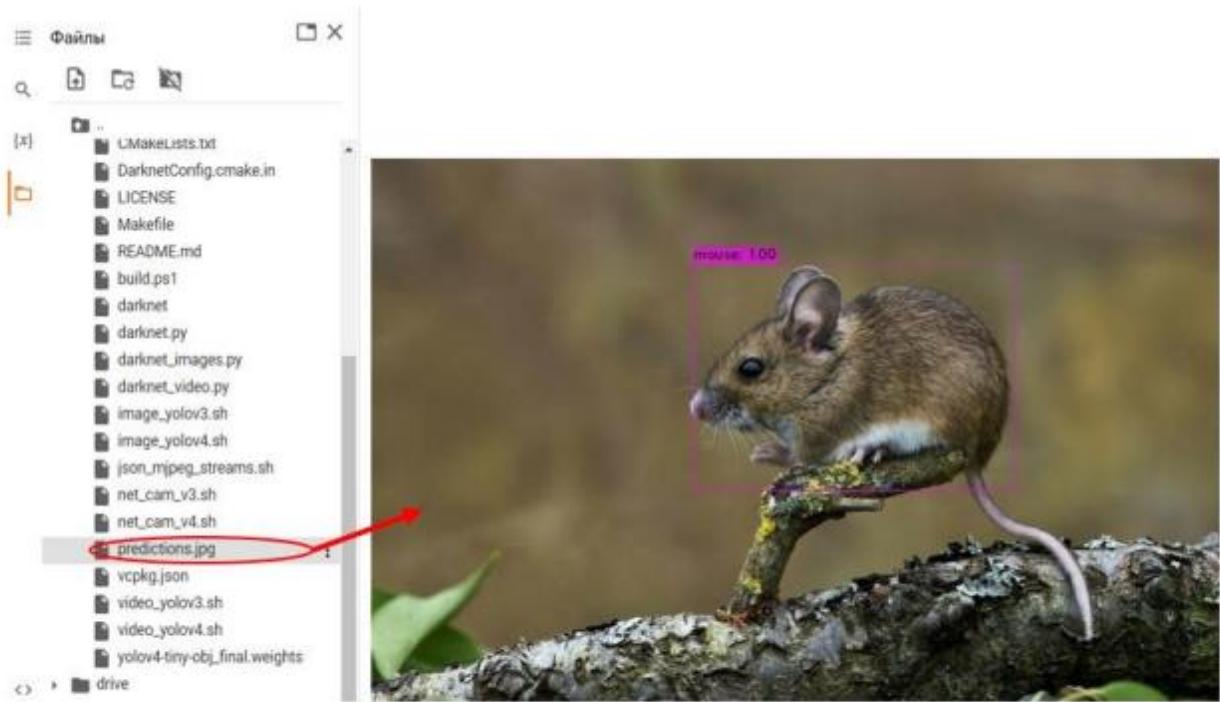


Рисунок 3.30. – Пример обнаружения объекта с использованием обученной СНС на облачном сервисе Google Colab

Перечень контрольных вопросов

- 1- Чем задача обнаружения объектов отличается от задачи классификации изображений и семантической сегментации?
- 2- Какие основные компоненты включает в себя сверточная нейронная сеть, используемая для обнаружения объектов?
- 3- Что такое «ограничивающая рамка» и какими параметрами она задаётся?
- 4- Как выполняется предобработка изображения перед подачей в сверточную нейронную сеть?

Установка CUDA Toolkit и cuDNN в Windows

CUDA Toolkit

- 1– загрузить версию CUDA с официального сайта: <https://developer.nvidia.com/cuda-toolkit> и распаковать;
- 2– в параметрах установки выбрать «Выборочно» и убедиться, что интеграция с MS VS включена (рисунок 3.31);

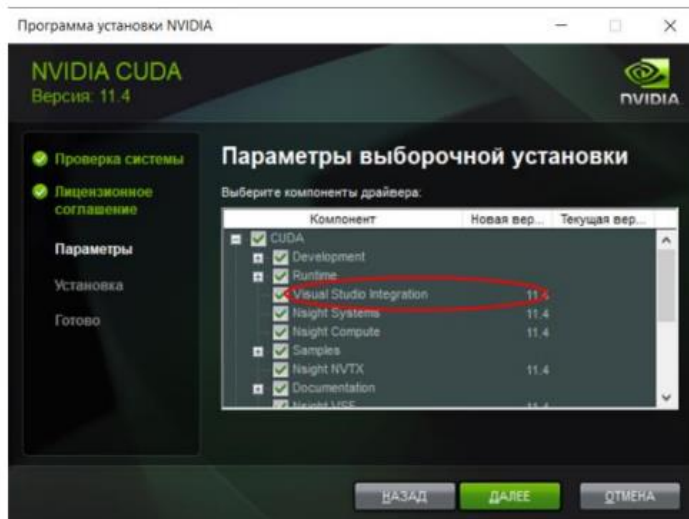


Рисунок 3.31. – Параметры выборочной установки CUDA

- 3- следовать инструкции до завершения установки;
- 4- убедиться, что инструментарий CUDA может правильно находить оборудование, поддерживающее CUDA, и взаимодействовать с ним. Для этого необходимо скомпилировать некоторые из включенных примеров программ или проверить версию CUDA;
- 5- версию CUDA Toolkit можно проверить, запустив `nvcc -V` в командной строке (рисунок 3.32).

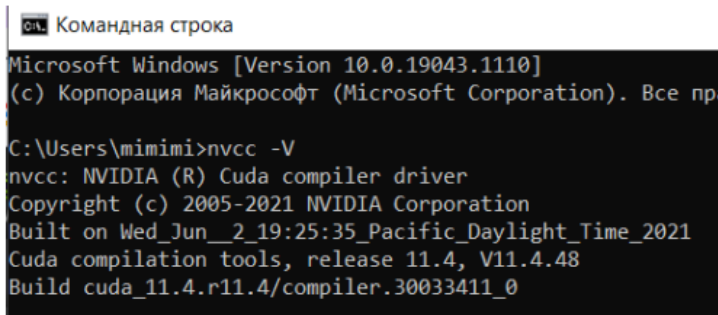


Рисунок 3.32. – Отображение версии CUDA

cuDNN

- 1– установить наиболее поздние драйверы на NVIDIA;
- 2– скачать cuDNN с официального сайта: <https://developer.nvidia.com/cudnn> (рисунок 3.33), но перед этим убедитесь, что зарегистрированы в программе разработчиков NVIDIA:

<https://developer.nvidia.com/developer-program>.

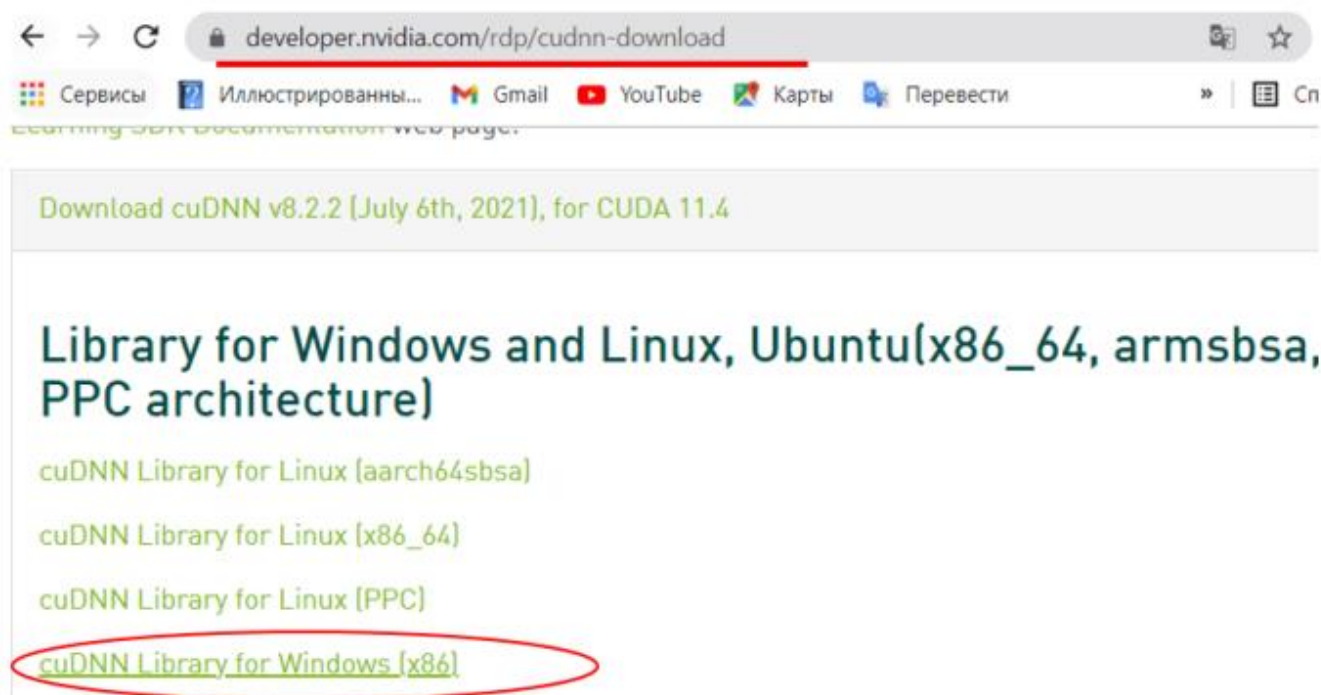


Рисунок 3.33. – Выбор cuDNN

- 1- распаковать архив;
- 2- перейти в каталог, содержащий файлы cuDNN. Скопировать указанные ниже файлы в каталог CUDA Toolkit. Следует обратить внимание, что пути на компьютерах пользователей могут отличаться.
 - а) копировать C:/cuda/bin/cudnn*.dll в C:/Program Files/NVIDIA GPU Computing Toolkit/CUDA/v11.4/bin;
 - б) копировать C:/cuda/include/cudnn*.h в C:/Program Files/NVIDIA GPU Computing Toolkit/CUDA/v11.4/include;
 - в) копировать C:/cuda/lib/x64/cudnn*.lib в C:/Program Files/NVIDIA GPU Computing Toolkit/CUDA/v11.4/lib/x64;
- 3- проверить, чтобы CUDA_PATH был добавлен в переменные среды.